Data & AI Builders Conference

Building modern data foundations for intelligent solutions together

Materialized Lake Views in Microsoft Fabric

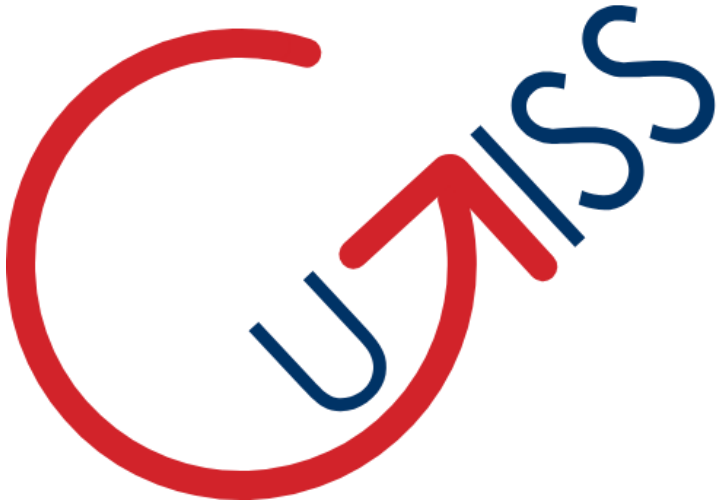
Trasformazioni dichiarative per il Lakehouse



Sponsors



Collaborators





About me

Solution Architect, Data & AI @ Lodestar S.p.A.

Working with SQL Server since 2005 edition

Working with Azure Data Platform since 2012

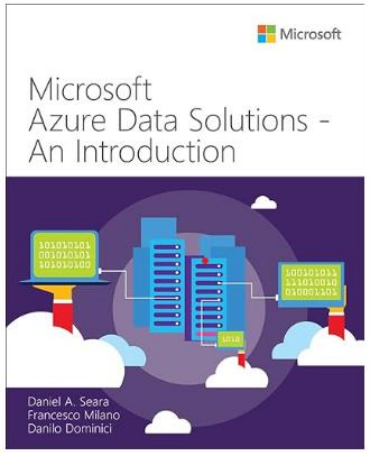
- ✓ Fabric Data Engineer Associate
- ✓ Fabric Analytics Engineer Associate

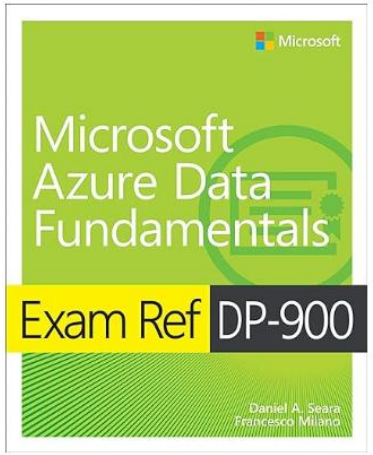
Email: francesco.milano@lodestar.it

LinkedIn: https://it.linkedin.com/in/fmilano

Medium: https://medium.com/@_fmilano_

icubed Lodestar





Scenario e problema: dalla SELECT al processo operativo

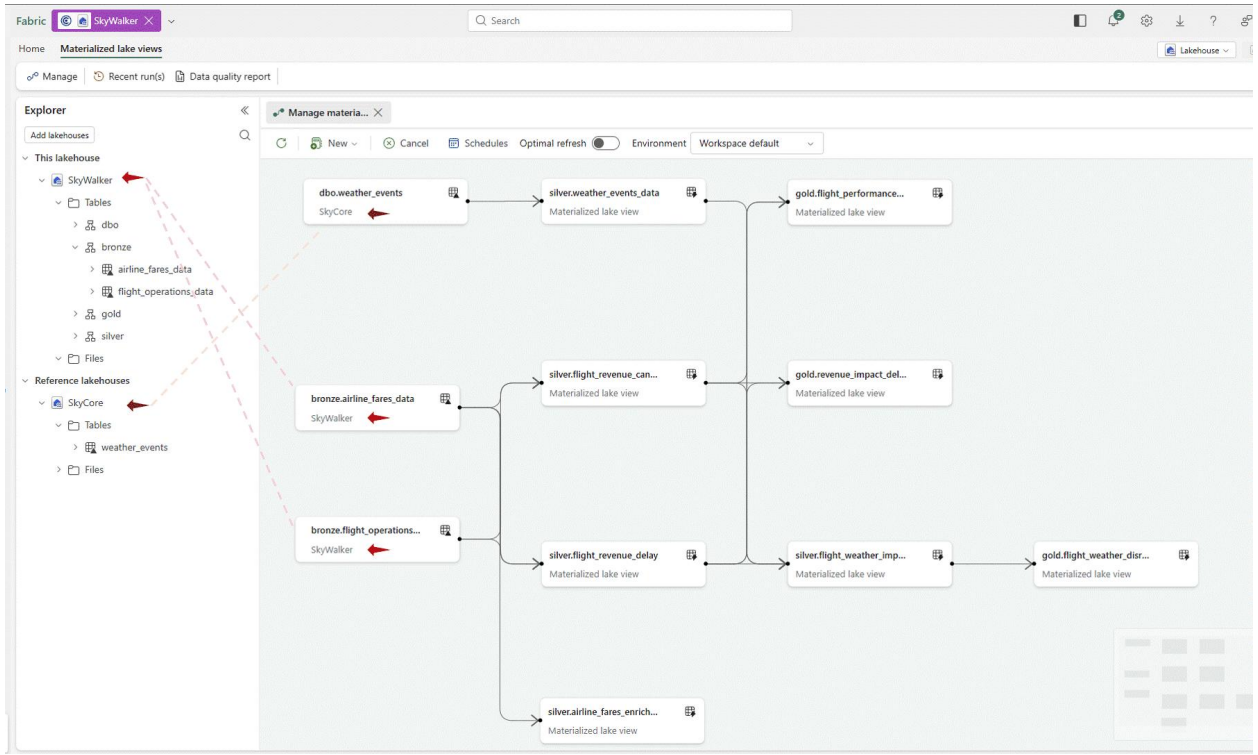
- **Scenario:** Lakehouse medallion (Bronze → Silver → Gold) con trasformazioni SQL su tabelle Delta che alimentano report e applicazioni.
- **Problema:** ogni trasformazione diventa un job. La query è solo una parte: attorno serve un sistema operativo da costruire e presidiare.
- **Costo reale:** non scrivere la query, ma mantenerla aggiornata, ordinata, osservabile e affidabile.
- **Esempio:** il livello Gold non è aggiornato perché Silver non ha completato. Dove si guarda?

Notebook/T-SQL	SELECT / CTAS e scrittura Delta
Pipeline	Orchestrazione, logica watermark / MERGE
Scheduling	Intervalli regolari o basato su eventi
Data quality	Check spesso non dichiarativi
Monitoring	Log e stato sparsi in più posti



Cos’è una Materialized Lake View

- Definita con una SELECT in Spark SQL
- Materializzata come tabella Delta nel Lakehouse
- Interrogabile come una normale tabella del Lakehouse
- Dipendenze, ordine di esecuzione e data quality gestiti da Fabric
- **Ciclo di vita:** Creazione → Refresh → Query → Monitoraggio



community.fabric.microsoft.com/.../materialized-lake-views-in-microsoft-fabric-generally-available/5172223

Spark SQL: definizione e comandi del ciclo di vita

Definizione

```
CREATE OR REPLACE MATERIALIZED LAKE VIEW
  silver.cleaned_order_data
(
  CONSTRAINT valid_quantity
    CHECK (quantity > 0) ON MISMATCH DROP
)
PARTITIONED BY (category)
COMMENT "Cleaned order data"
TBLPROPERTIES ("layer" = "silver")
AS SELECT p.productID, p.category,
          o.orderDate, o.quantity
FROM bronze.products p
INNER JOIN bronze.orders o
  ON p.productID = o.productID
```

Comandi del ciclo di vita

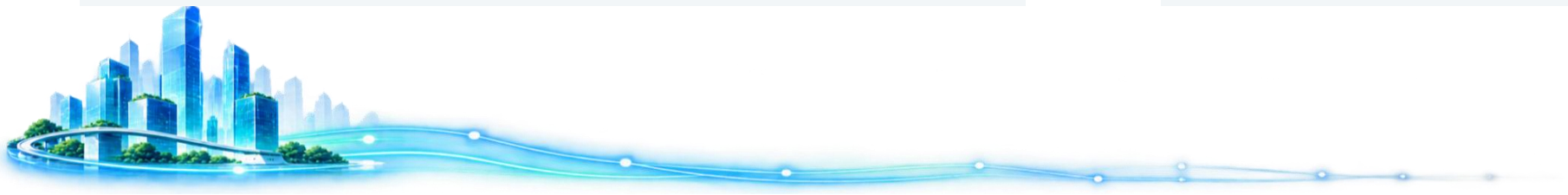
```
-- Elenca le MLV dello schema silver
SHOW MATERIALIZED LAKE VIEWS IN silver;

-- Mostra la definizione della MLV
SHOW CREATE MATERIALIZED LAKE VIEW
  silver.cleaned_order_data;

-- Rinomina la MLV
ALTER MATERIALIZED LAKE VIEW
  silver.cleaned_order_data
  RENAME TO silver.orders_v2;

-- Ricalcolo completo (Full refresh)
REFRESH MATERIALIZED LAKE VIEW
  silver.orders_v2 FULL;

-- Elimina la MLV
DROP MATERIALIZED LAKE VIEW
  silver.orders_v2;
```



PySpark (Preview): cosa cambia rispetto a Spark SQL

PYSPARK: PREVIEW

- **Spark SQL (GA):** DDL dichiarativo
- **PySpark:** @fmlv.materialized_lake_view in un notebook; supporto a librerie esterne e UDF personalizzate
- **Refresh:** solo Full o No refresh, nessun Incremental
- **Dipendenza:** il notebook deve essere disponibile durante i refresh
- **Data quality per colonna:** suite Python (@fmlv.dq_suite, validate, analyze) su Deequ, separate dalla definizione

```
# Definizione PySpark (Preview)
@fmlv.materialized_lake_view(
    name="silver.cleaned_order_data",
    comment="Cleaned order data")
# Vincolo di data quality
@fmlv.check(
    name="valid_quantity",
    condition="quantity > 0",
    action="DROP")
def cleaned_order_data():
    return spark.sql("""
        SELECT p.productID, p.category,
               o.orderDate, o.quantity
        FROM bronze.products p
        INNER JOIN bronze.orders o
          ON p.productID = o.productID""")
```

learn.microsoft.com/.../data-quality



Ingestione di file dal livello Bronze

PREVIEW

Parametri

- **Origine:** cartella OneLake fisica, shortcut a una cartella o singolo file
- **Formati:** csv | parquet
- **schema_mode:** DYNAMIC / FIXED
- **refresh_mode:** APPEND_ONLY / FULL / MIRROR
- **__filepath__** per tracciare il file di origine

USING OneLake_Files

```
CREATE MATERIALIZED LAKE VIEW
bronze.raw_orders
USING OneLake_Files
OPTIONS (
  'format' = 'csv',
  'path'   = 'abfss://<workspace>@<host>/
              <lakehouse>/Files/orders/',
  'header' = 'true'
)
TBLPROPERTIES (
  'schema_mode'  = 'DYNAMIC',
  'refresh_mode' = 'APPEND_ONLY'
);
```

learn.microsoft.com/.../create-materialized-lake-view



Scenari forti e anti-scenari

Scenari adatti

- **Aggregazioni consultate spesso:** il risultato persistito evita ricalcoli a ogni query
- **Join complessi e riutilizzabili:** definiti una sola volta e letti da molti
- **Data quality dichiarativa e uniforme:** vincoli CHECK applicati a ogni refresh
- **Dataset di reporting multi-sorgente:** dipendenze e ordine gestiti da Fabric
- **Flussi Bronze → Silver → Gold in SQL:** trasformazioni dichiarative a livelli

Scenari non adatti

- **Query una tantum:** la materializzazione non ripaga il costo
- **Chiamate API, inferenza ML, Python complesso:** logica procedurale, fuori dal modello dichiarativo
- **Workflow procedurali:** meglio una pipeline o un notebook
- **Streaming sotto il secondo:** lo schedule minimo effettivo è di 5 minuti
- **Costrutti SQL non supportati:** la trasformazione non è esprimibile come MLV



Funzionalità GA e Preview a novembre 2026

Funzionalità	Stato	Indicazione
Nucleo MLV, Spark SQL, lineage gestito, schedule, monitoraggio	**GA**	
Authoring PySpark (decorator fmlv)	**Preview**	Solo Full o No refresh; vedi nella slide 8
Ingestione di file con USING OneLake_Files	**Preview**	CSV e Parquet; ancora Preview nel blog ufficiale del 13 ottobre 2026
Refresh attivato da eventi	**Preview**	Eventi OneLake, eventi dei job Notebook, eventi dei job Pipeline
REFRESH_HINT … UNIQUE per update e delete	**Preview**	Fabric non valida l'unicità in esecuzione
Utility per notebook refreshMlv	**Preview**	Sviluppo, test, troubleshooting e uso ad hoc; non equivale all'orchestrazione gestita

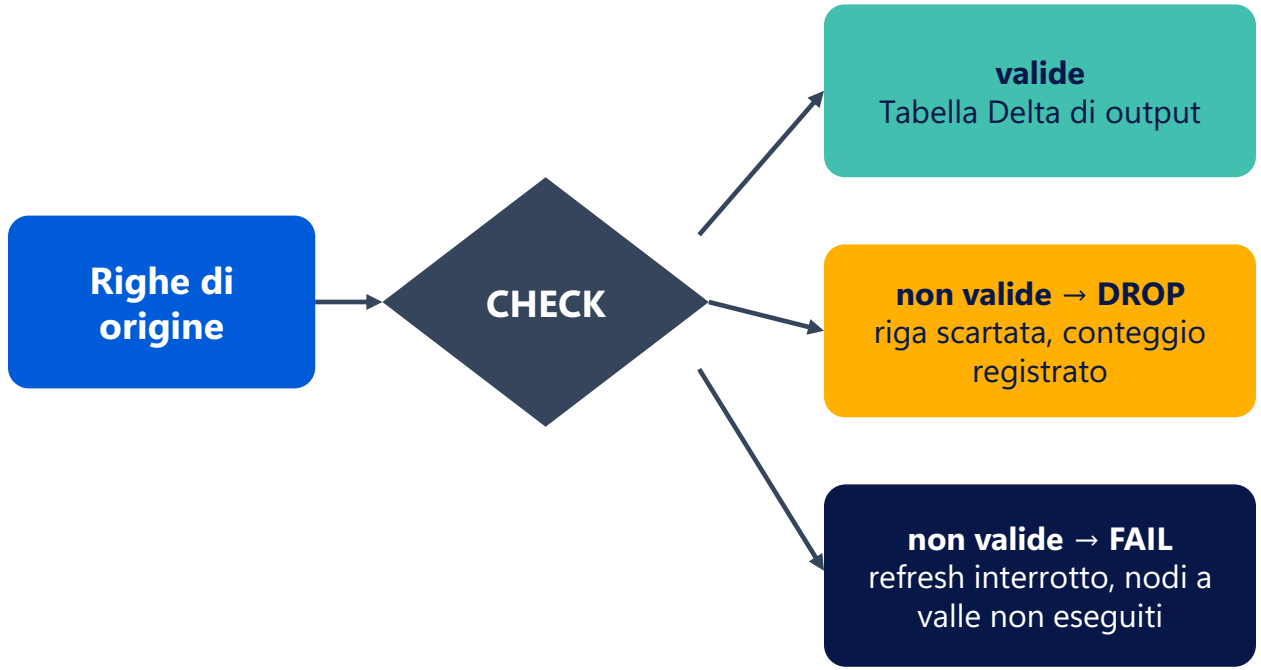
CDF: abilitato di default sull'output delle nuove MLV, ma l'Incremental refresh richiede CDF su tutte le tabelle Delta di origine.



Data quality a livello di riga

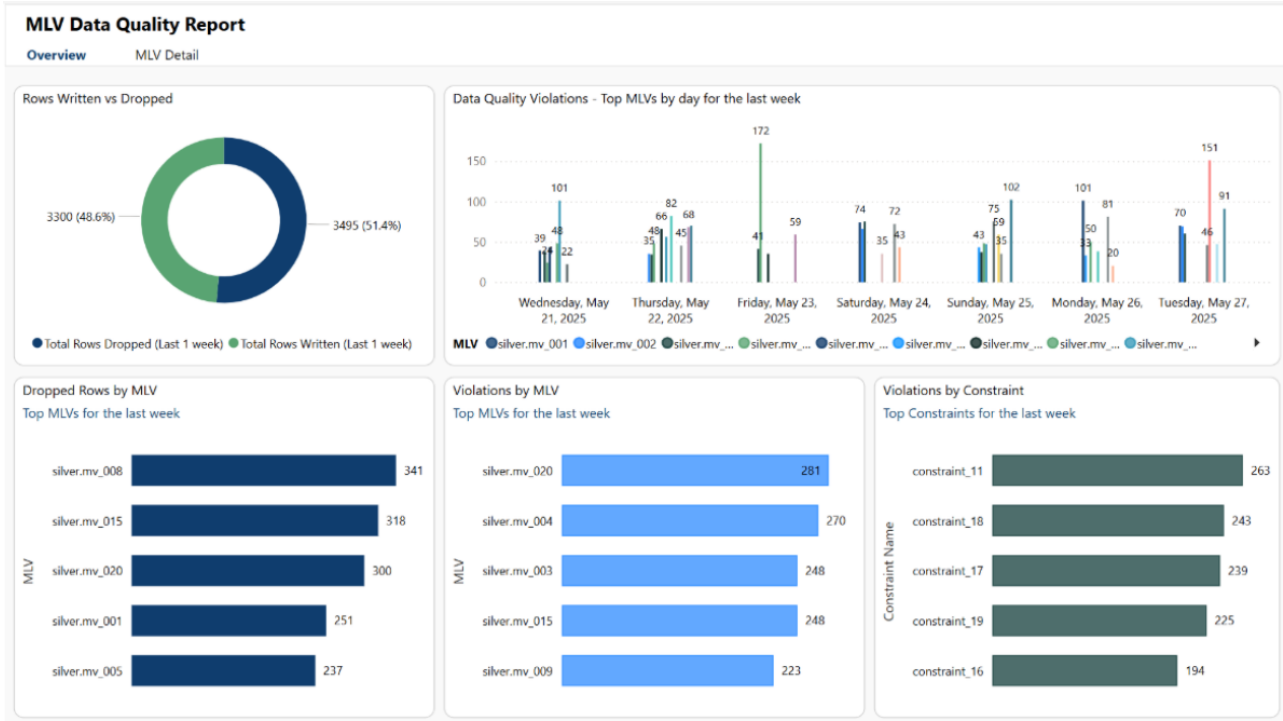
- Vincolo CHECK booleano, valutato su ogni riga
- **ON MISMATCH FAIL** è il comportamento predefinito
- **DROP** rimuove la riga e ne registra il conteggio
- Se coesistono DROP e FAIL, prevale FAIL
- Supportate le funzioni Spark/SQL native e deterministiche

```
-- Sintassi del vincolo CHECK
-- ON MISMATCH: DROP | FAIL (default)
CREATE MATERIALIZED LAKE VIEW silver.orders
(
  CONSTRAINT valid_quantity
    CHECK (quantity > 0) ON MISMATCH DROP
)
AS SELECT * FROM bronze.orders;
```



Report sulla data quality

- **A livello di riga:** CONSTRAINT … CHECK con ON MISMATCH DROP o FAIL
- **DROP:** il conteggio delle righe scartate è visibile nel lineage
- **FAIL:** il refresh si ferma e le MLV a valle non vengono eseguite
- **Cronologia delle run:** mostra lo stato di data quality di ogni run
- **Report Power BI sulla data quality:** pagine Overview e MLV Detail

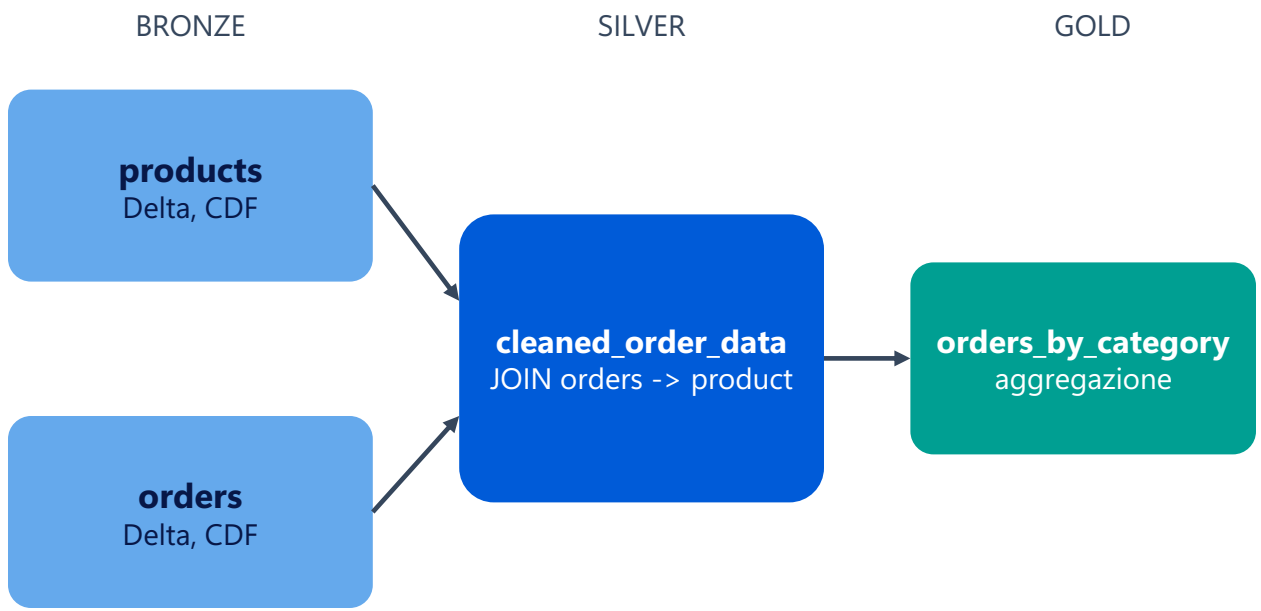


learn.microsoft.com/.../data-quality-reports

Demo: scenario

- MLV Silver con join
- MLV Gold aggregata
- Tabelle Delta persistite e lineage
- Modifica della sorgente e refresh
- Policy di refresh, dettaglio delle run

- Data Quality



Demo

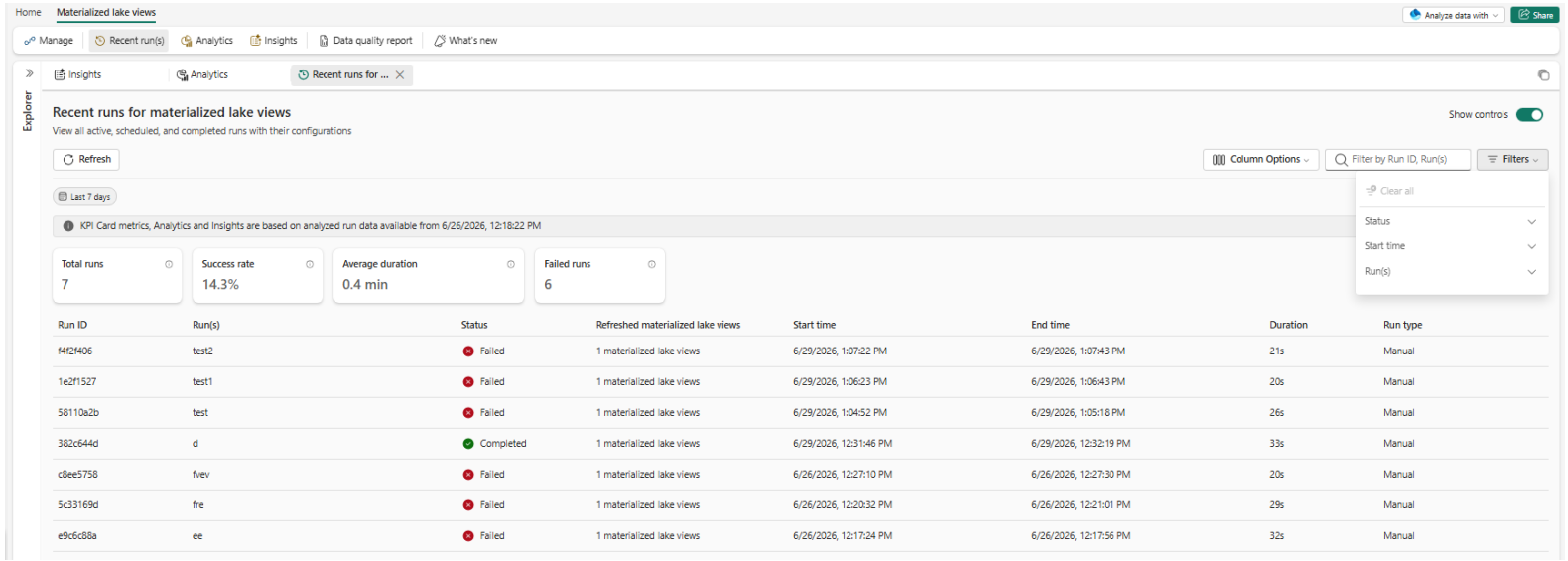
LIVE DEMO — Bronze → Silver → Gold



Lodestar

Monitoraggio: dal trend al log di Spark

- **Recent runs:** aggiornamento, stato e dettaglio di ogni run
- **Analytics:** andamento delle run, errori più frequenti, stato e distribuzione delle policy di refresh
- **Insights:** raccomandazioni in ordine di priorità: Risk, Diagnosis, Optimize
- **Monitor hub:** vista centralizzata; tipo di job RefreshMaterializedLakeViews
- **Nodo in errore:** Error Code, Message, log, Spark UI

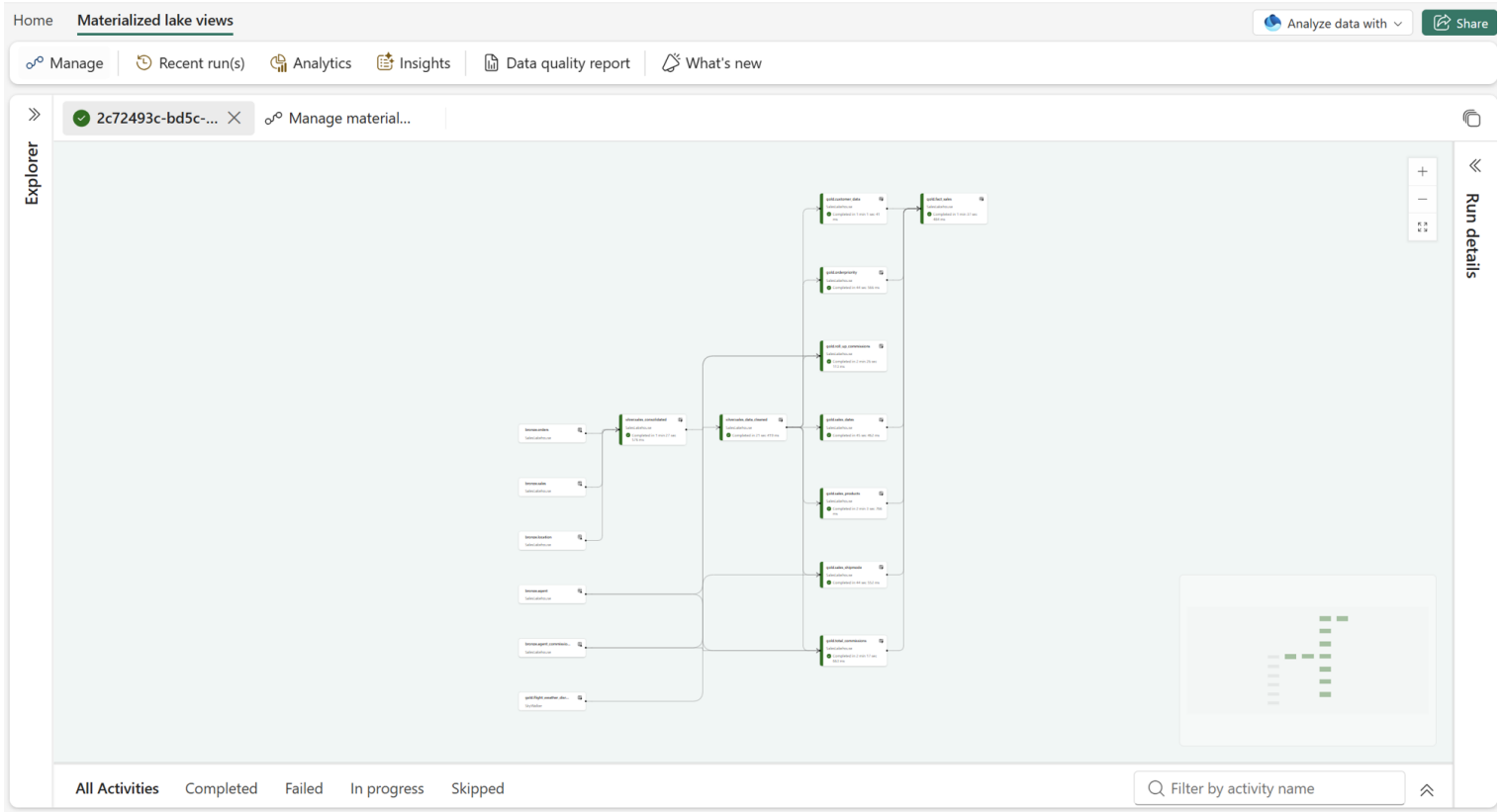


learn.microsoft.com/.../run-history

Lodestar

Lineage ed Extended lineage

- Prima le MLV a monte, poi quelle a valle
- I rami indipendenti procedono in parallelo
- **Lineage standard:** Lakehouse corrente + dipendenze a monte immediate
- **Extended lineage:** ricorsiva, tra più Lakehouse
- Dipendenza mancante o inaccessibile → nodo in stato faulted



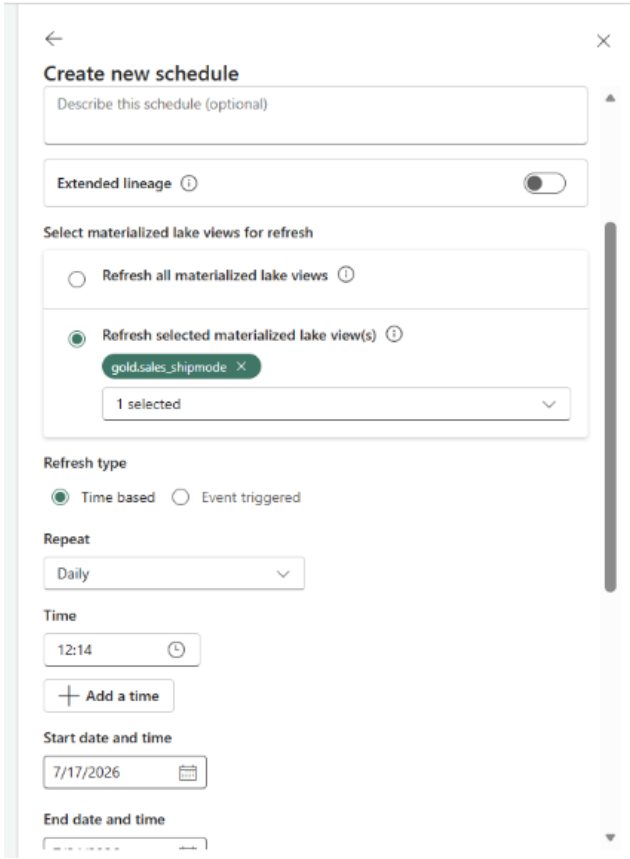
learn.microsoft.com/.../view-lineage

Lodestar

Pianificazione e orchestrazione

REFRESH DA EVENTI: PREVIEW

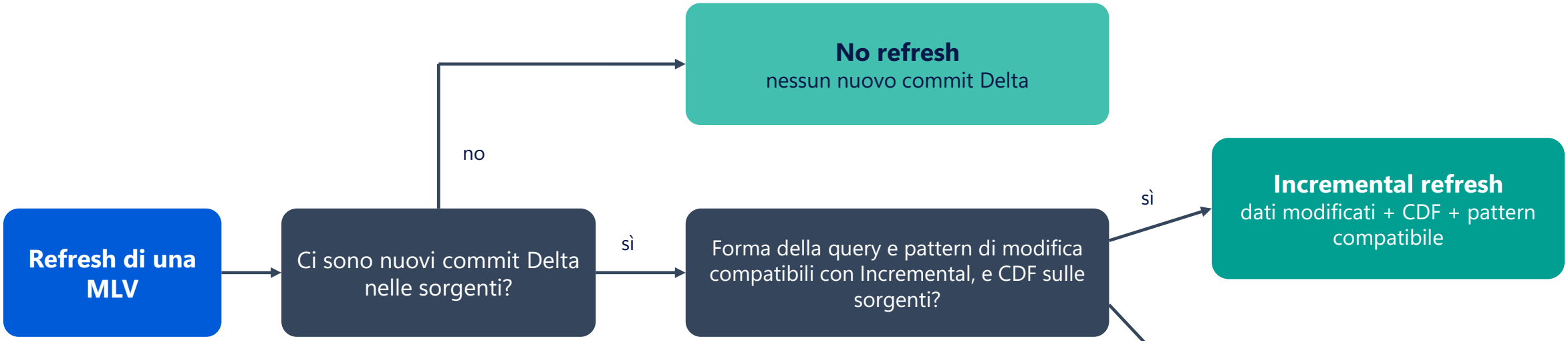
- Più schedule, per MLV selezionate o per l'intero grafo
- **Intervallo minimo effettivo:** 5 minuti
- **Refresh attivato da eventi (Preview)**
- Schedule tra più Lakehouse tramite Extended lineage
- Activity di pipeline: aggiorna tutte le MLV del Lakehouse; nessun supporto per SPN e workspace identity



learn.microsoft.com/.../schedule-lineage-run

Optimal refresh: No, Incremental o Full

REFRESH_HINT UNIQUE: PREVIEW



CDF è necessario ma non sufficiente: non coincide con Incremental refresh.

Update e delete: REFRESH_HINT … UNIQUE (Preview)
Fabric non valida l'unicità dichiarata

Troubleshooting e limitazioni

Troubleshooting

1. Verificare stato di run e rami, e nodi a valle saltati (Skipped)
2. Leggere Error Code, Message e dettagli del nodo
3. Aprire la Spark UI per errori di driver ed executor
4. Verificare CDF, schema delle sorgenti, permessi e forma della query
5. Usare Full refresh per diagnosi o rielaborazione

Limitazioni

- Nessun DML (INSERT, UPDATE, DELETE) sulla MLV
- Nessun time travel nella definizione
- Nessuna UDF SQL nella SELECT
- Nomi di schema interamente in maiuscolo non supportati



Consumi e modello di costo

Voce di costo	Cosa osservare	Strumento
Refresh MLV	Calcolo Spark / Data Engineering	Capacity Metrics app
Optimal refresh	Nessun costo aggiuntivo; fatturazione sul calcolo utilizzato	Capacity Metrics app
Autoscale billing	Calcolo Spark fatturato	Azure Cost Analysis
Risultato Delta persistito	Spazio di storage occupato dalla tabella Delta	Da includere nel TCO
Alternativa con Pipeline	Data Orchestration, Data Movement e workload invocati	Capacity Metrics app

Per confrontare: stesso volume di dati, stesso SLA, stessa gestione di errori e retry, stesso impegno di sviluppo.



MLV vs Pipeline + Notebook/Spark job

Dimensione	Materialized Lake View	Pipeline + Notebook/Spark job
Modello	Risultato desiderato dichiarativo	Procedurale, basato su activity
Dipendenze	Dedotte dai riferimenti	Orchestrate esplicitamente
Pianificazione	Schedule multipli gestiti / eventi (Preview)	Trigger di Pipeline e control flow
Refresh	No / Incremental / Full deciso da Fabric	Watermark, MERGE e strategia implementati dal team
Persistenza	Tabella Delta gestita dalla MLV	Tabella scritta dal job
Complessità	Meno codice di raccordo	Più controllo, più artefatti
Osservabilità	Recent runs, Analytics, Insights, lineage	Monitoraggio Pipeline + Spark UI + log personalizzati
Troubleshooting	Drill-down su run, nodo, errore e Spark	Activity per activity
Flessibilità	Confini del SQL (limitazioni documentate)	Python, API, ML, effetti collaterali
Data quality	Constraint row-level e Data quality report integrati	Framework personalizzato e componibile
Consumi	Calcolo Spark per il refresh	Orchestrazione + spostamento dati + calcolo dei workload invocati



Linee guida decisionali

Decision gate

1. Il processo si scrive come SELECT dichiarativa riutilizzabile, senza logica procedurale?
2. Salvare il risultato come tabella dà un vantaggio concreto (riuso, prestazioni)?
3. Sorgenti e query permettono il refresh che serve (es. incrementale)?
4. Tempi di aggiornamento e gestione errori di Fabric bastano al caso d'uso?
5. Data quality, permessi, monitoraggio e ciclo di vita sono coperti senza sviluppi extra?
6. Il costo totale (capacità, sviluppo, operatività) è inferiore a Pipeline + Notebook?

Esiti possibili

MLV
Tutte le risposte SÌ: trasformazione dichiarativa, refresh gestito da Fabric

Ibrido
NO su 3, 4 o 5: Pipeline come macro-orchestratore, MLV per il sottografo dichiarativo

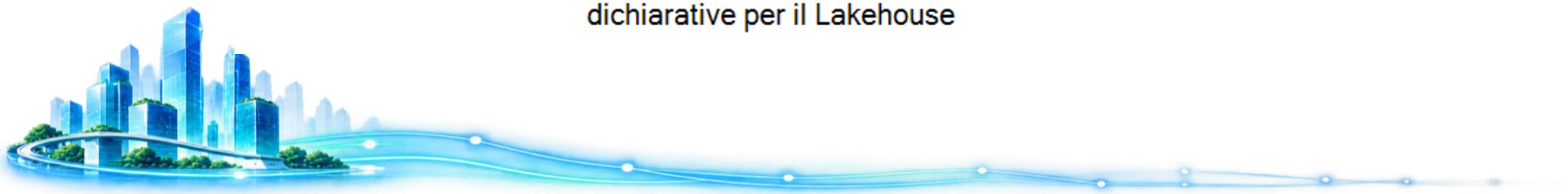
Pipeline o Notebook
NO su 1, 2 o 6: logica procedurale, API, ML oppure costo più alto con MLV





Materialized Lake Views in Microsoft Fabric: trasformazioni dichiarative per il Lakehouse

THANK YOU!!!



Lodestar

Riferimenti

Learn: authoring e data quality

- overview-materialized-lake-view
- get-started-with-materialized-lake-views
- tutorial
- create-materialized-lake-view
- create-materialized-lake-view-pyspark
- data-quality
- data-quality-reports

Learn: operatività

- view-lineage
- schedule-lineage-run
- refresh-materialized-lake-view
- optimal-refresh-handling-deletes-updates
- run-history
- analytics
- insights
- monitor-materialized-lake-views

Altre fonti

- Learn: refresh-materialized-lake-view-activity
- Learn: billing-capacity-management-for-spark
- Learn: pricing-pipelines
- Blog: MLV Generally Available
- Blog: File data ingestion (Preview)
- Blog: Mastering declarative data transformations
- GitHub: fabric-samples, MLV tutorial
- Fabric Jumpstart: materialized-lake-views

